

PROGRAMMING 2B

PROG6212

PART 1
GROUP 1

| ST10475253
| **Sandile Nhlapo**



TABLE OF CONTENTS

Contents

TABLE OF CONTENTS	2
SECTION A.....	3
SECTION B.....	5
SECTION C	10
Full SQL Query	18

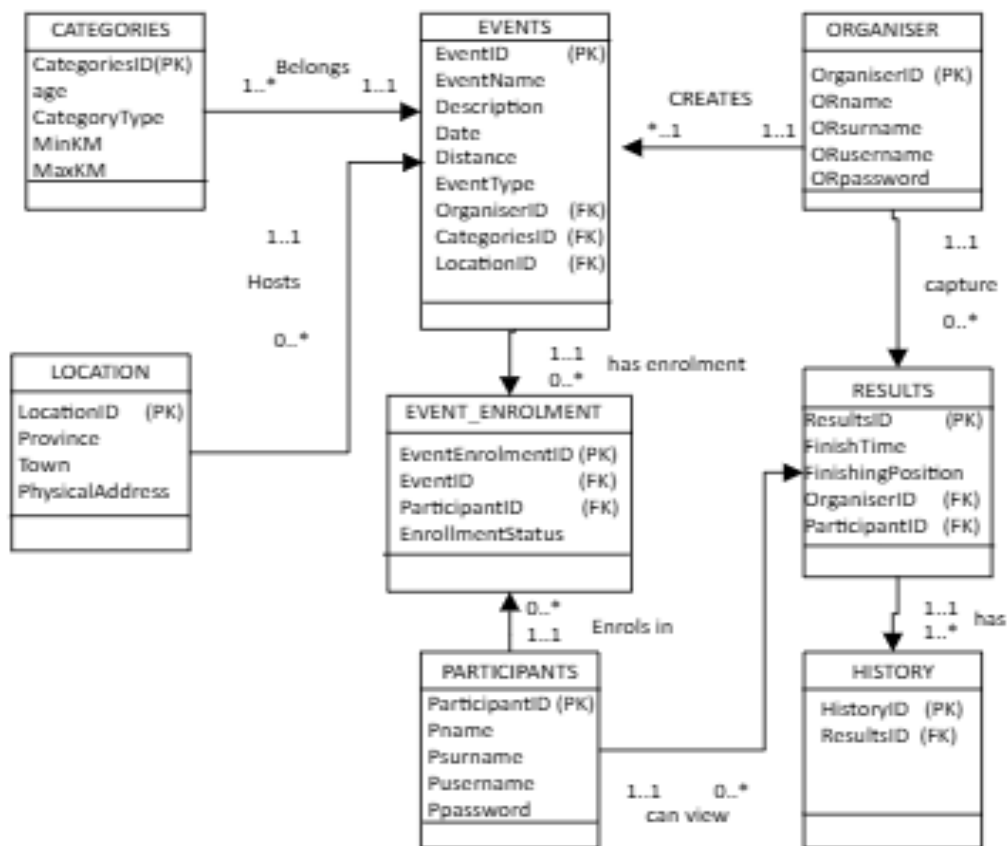
YouTube Link:

<https://youtu.be/-7yrWlNhY0A>

GitHub Link:

<https://github.com/Sandile081/ST10475253-SB-NHLAPO-PROG6212>

SECTION A



This RaceDay Event Management System ERD was designed to represent all the key elements involved in the race management process without having redundant data. This model provides an easy to understand representation of how all the relevant components interact with each other. The model includes 8 key components: Organiser, Participant,

Location, Category, Event, Event Enrolment, Result and History and each component is defined with its primary and foreign keys to provide a structured and logical relationship.

The Organiser component will contain information regarding those individuals creating and managing events and each individual may be able to create many events. An organiser can therefore create one event per category. To support age groups and distance divisions the number of categories available to each event can be unlimited. As there could potentially be many locations in which events occur the location of each event is contained within a separate table from the event itself to prevent duplication and inconsistencies. The many-to-many relationship between Participant and Event is resolved through the use of an associative Entity called Event Enrolment. Event Enrolment contains the detail of which participant has enrolled into which event as well as whether the participant has been successfully enrolled into the event.

Once an event is complete, the Results table contains the participant's completion time and position in relation to other participants. The Results table links to both the participant and the organisers responsible for recording the result. The History table acts as an immutable archive of previous results for auditing and long-term tracking purposes. The correct application of cardinality is demonstrated throughout the model for example; one organiser can produce many events, one event can have many categories, one participant can produce many enrolments, and one enrolment can have either zero or one result.

SECTION B

HTTP METHOD	ROUTE	DESCRIPTION	ROLE REQUIRED	REQUEST BODY	EXPECTED RESPONSE
POST	/api/organiser	Creates Records of an organiser. This is linked the public to Organiser to input their name, surname, username and password	Public	{ ORname, ORsurname, ORusername, ORpassword }	201 Created – organiser records with name, surname, username and password. 409 Conflict – username already exists
POST	/api/participants	Creates records of a participant. This is linked to the public to participants to input their name, surname, username and password	Public	{ Pname, Psurname, Pusername, Ppassword }	201 Created – participant records with name, surname, username and password. 409 Conflict – username already exists
POST	/api/events	Creates records of a events. This is linked to the Logged in member. Organiser can able to create, edit and delete an event.	Members (Organiser)	{ EventName, Description, Date, Distance, EventType, OrganiserID, CategoriesID, LocationID }	201 created – Organiser records with the event name, description, date, distance, event type, organiserID, categoriesID, LocationID. 200 OK – the event was successfully created.
POST	/api/results	Create records of a results. This linked in organisers. Organiser are able to add the following ResultsID, FinishTime, FinishingPosition, OrganiserID, ParticipantID	Member (Organiser)	{ FinishTime, FinishingPosition, OrganiserID, ParticipantID }	201 created – Organisers records with the finishing time, finishing Position, OrganiserID and ParticipantID. 200 OK – the event was successfully created.
POST	/api/category	Create the records of the category. This linked in Organiser are	Member (Organiser)	{ age, CategoryType, MinKM, MaxKM }	201 created – Organisers records with the age, CategoryType,

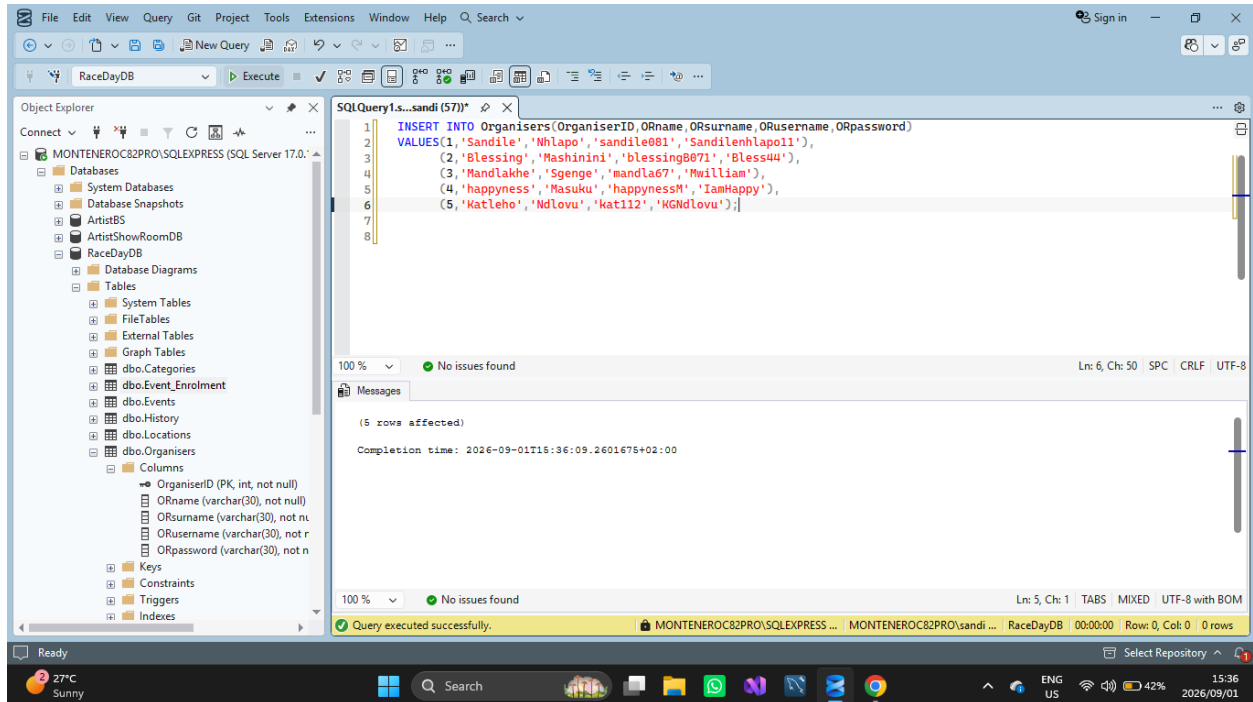
		able to create categories by adding age, CategoryType, MinKM and MaxKM		}	MinKM and MaxKM. 200 OK – the category was successfully created
POST	/api/location	Create the records of location. This linked in Organiser to able to create Province, Town and PhysicalAddress	Member (Organiser)	{ Province, Town, PhysicalAddress }	201 created – Organisers records with the Province, Town and PhysicalAddress . 200 OK – the location was successfully created
GET	/api/events	Retrieve all records of events. This is linked to organiser and participant to able to view EventID, EventName, Description, Date, Distance, EventType, OrganiserID, CategoriesID and LocationID.	Member	None	200 OK – the events was successfully displayed.
GET	/api/results/{ResultsID}	Retrieve specific records of results. Organiser and participants can to view specific results ResultsID, FinishTime, FinishingPosition, OrganiserID and ParticipantID	Member	None	200 OK – the events was successfully displayed. 404 Not Found – the results does not exist.
GET	/api/participants/{ParticipantID}/results	Retrieve specific records of results. participants can view their specific results ResultsID, FinishTime, FinishingPosition, OrganiserID and	Participant	None	200 OK – the events was successfully displayed. 404 Not Found – the results does not exist.

		ParticipantID			
GET	/api/history	Retrieve all records of history. This linked in organiser to able to view HistoryID and ResultsID.	Organiser	None	200 OK – the history was successfully displayed. 404 Not Found – the History does not exist.
GET	/api/participant/{ParticipantID}/history	Retrieve specific the record of History for that specific participant. This is Linked to member (participant) to view HistoryID and ResultsID.	Participants	None	200 OK – the history was successfully displayed. 404 Not Found – the History does not exist.
GET	/api/location	Retrieve all the record of Location. This is Linked to member to view LocationID, Province, Town and PhysicalAddress.	Members	None	200 OK – the location was successfully displayed. 404 Not Found – the location does not exist.
GET	/api/location/{LocationID}	Retrieve specific the record of Location. This is Linked to member to view LocationID, Province, Town and PhysicalAddress.	Members	None	200 OK – the location was successfully displayed. 404 Not Found – the location does not exist.
GET	/api/event_enrolment	Retrieve all records of event_enrolment. This is Linked to members to view EventEnrolmentID, EventID, ParticipantID and EnrollmentStatus.	Members	None	200 OK – the event enrolment was successfully displayed. 404 Not Found – the event enrolment does not exist.

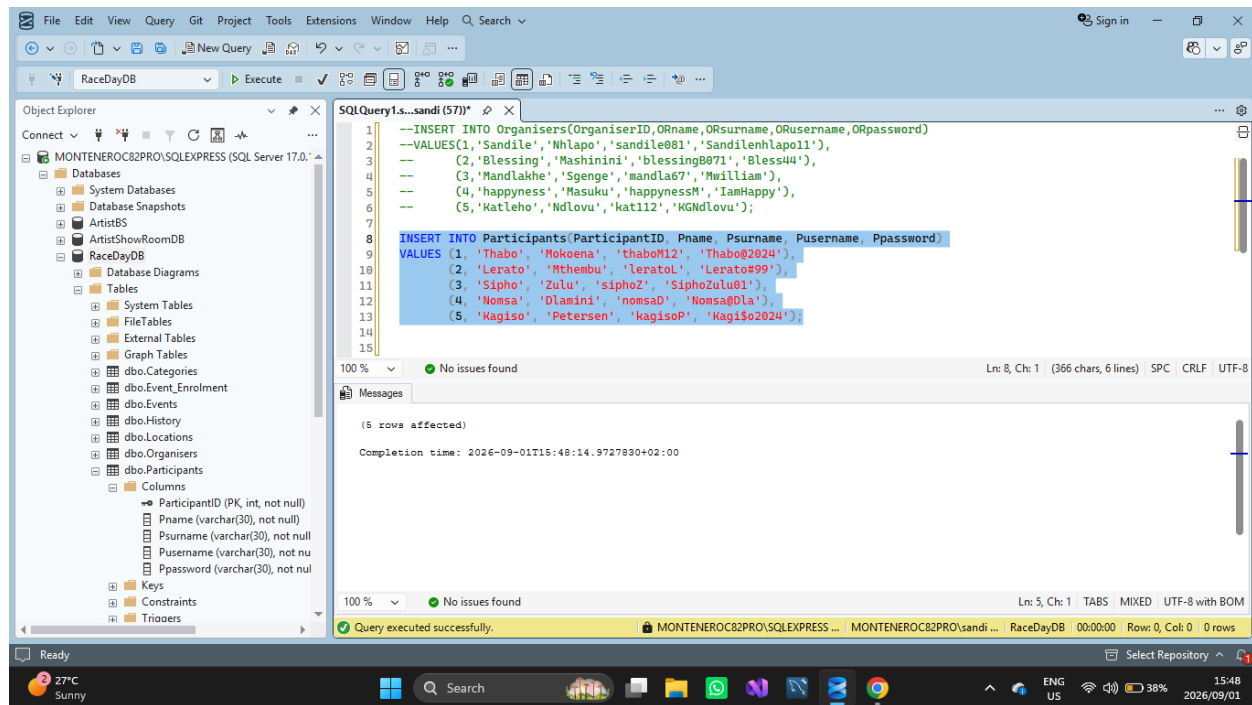
GET	/api/event_enrolment/{EventEnrolmentID}	Retrieve specific records of event_enrolment. This is Linked to members to view EventEnrolmentID, EventID, ParticipantID and EnrollmentStatus.	Members	None	200 OK – the event enrolment was successfully displayed. 404 Not Found – the event enrolment does not exist.
PUT	/api/organiser/{OrganiserID}	Update specific records of organiser. This linked to a specific Organiser to update ORname, ORsurname, ORusername and ORpassword	Organiser	{ ORname, ORSurname, ORusername, ORpassword }	201 Created – organiser records with name, surname, username and password.
PUT	/api/participant/{ParticipantID}	Update specific records of organiser. This linked to a specific participant to update Pname, Psurname, Pusername and Ppassword.	Participant	{ Pname, Psurname, Pusername, Ppassword }	201 Created – participants records with name, surname, username and password.
PUT	/api/event/{EventID}	Update specific records of event. This is Linked to a organiser to update EventName, Description, Date, Distance, EventType, OrganiserID, CategoriesID and LocationID	Organiser	{ EventID, EventName, Description, Date, Distance, EventType, OrganiserID, CategoriesID, LocationID }	201 Created – participants records with EventName, Description, Date, Distance, EventType, OrganiserID, CategoriesID and LocationID
PUT	/api/categories/{CategoriesID}	Update specific records of Categories. This linked to update age, CategoryType, MinKM and MaxKM	Organiser	{ Age, CategoryType, MinKM, MaxKM }	201 Created – participants records with Age, CategoryType, MinKM and MaxKM

		MaxKM.			
PUT	/api/location/{LocationID}	Update specific records of location. This linked to organiser to update LocationID, Province, Town and PhysicalAddress	Organiser	{ Province, Town, PhysicalAddress }	201 Created – participants records with Province, Town and PhysicalAddress
PUT	/api/result/{ResultsID}	Update specific records of results. This linked to organiser to update FinishTime, FinishingPosition, OrganiserID and ParticipantID	Organiser	{ FinishTime, FinishingPosition, OrganiserID and ParticipantID }	201 Created – participants records with FinishTime, FinishingPosition, OrganiserID and ParticipantID
DELETE	/api/events/{EventID}	Delete specific records of events. This linked to organiser to delete EventName, Description, Date, Distance, EventType, OrganiserID, CategoriesID and LocationID.	Organiser	{ EventName, Description, Date, Distance, EventType, OrganiserID, CategoriesID and LocationID }	200 OK – the location was successfully Deleted

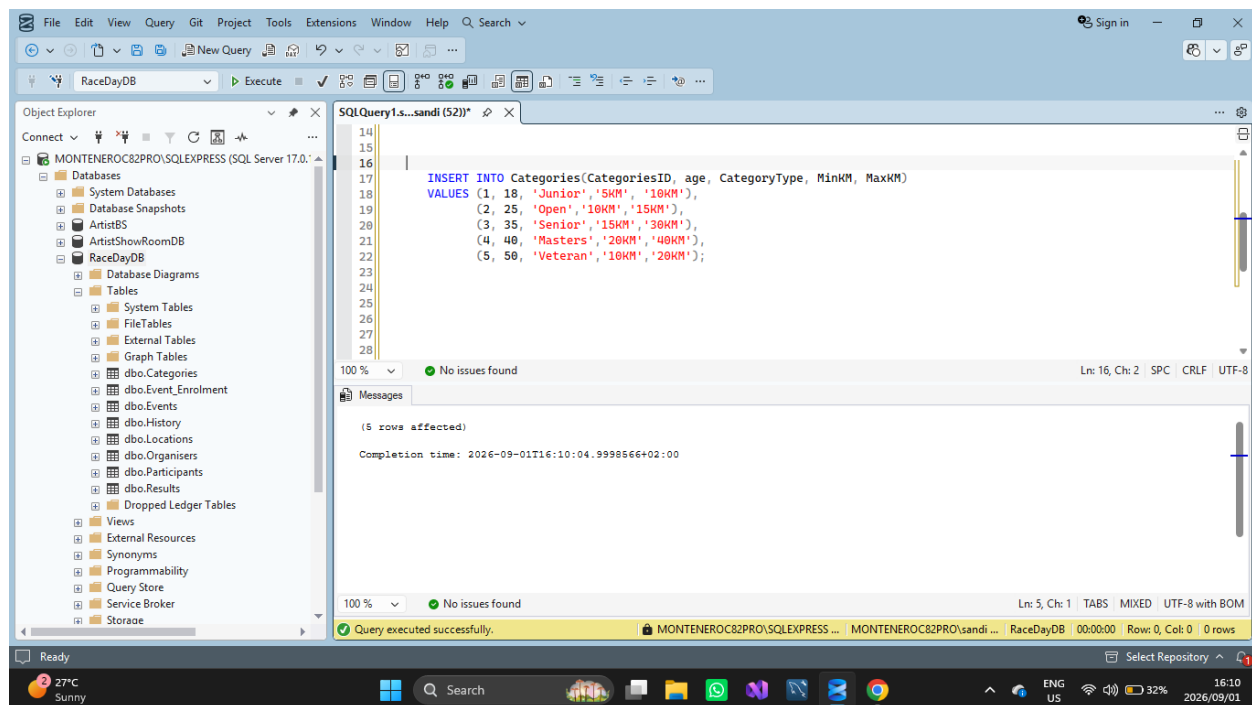
SECTION C



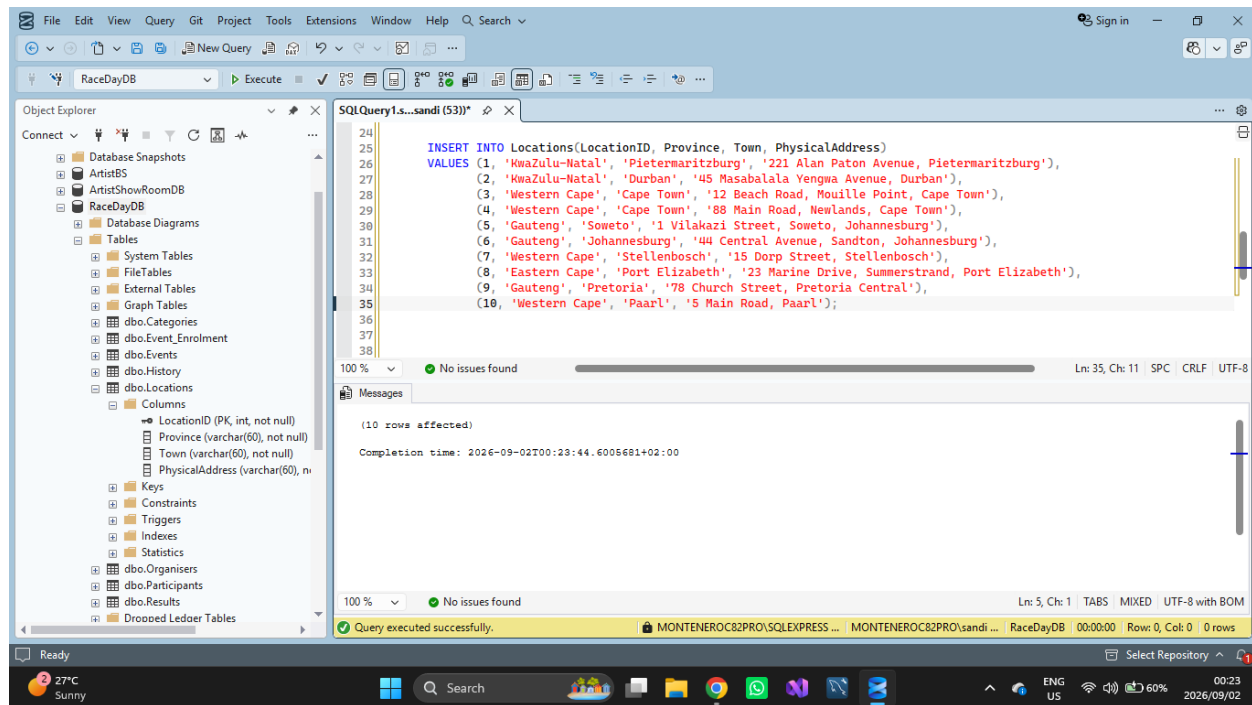
The five organisers entered above will be able to register on the RaceDay site and then use their login credentials to access their personal account. Their primary responsibility is to create events on the RaceDay site, including defining details such as event name, description, date, distance, type, and location. Each event also has multiple categories defined by the organiser such as 'under 20', 'senior', '10km', '21km' that participants can choose from when enrolling in that particular event. The organiser will be able to see every participant who has enrolled in an event they have created along with the participant's status at that time. After an event has taken place, the organiser is responsible for recording each participant's results, for example how long it took them to complete the course, and where they finished. These results will automatically populate the Results table and be available to the participants. As stated previously, organisers are responsible for setting up and administering events but will NOT be participating in those events. Therefore, they will have complete authority to make changes to their events or delete them if necessary. They will also be able to edit details regarding an event. This includes making any changes necessary prior to the event being held and publishing the final results after the event. The organisers are key to the success of this project, and their responsibilities include establishing the correct settings for an event, registering participants for an event, and reporting final results.



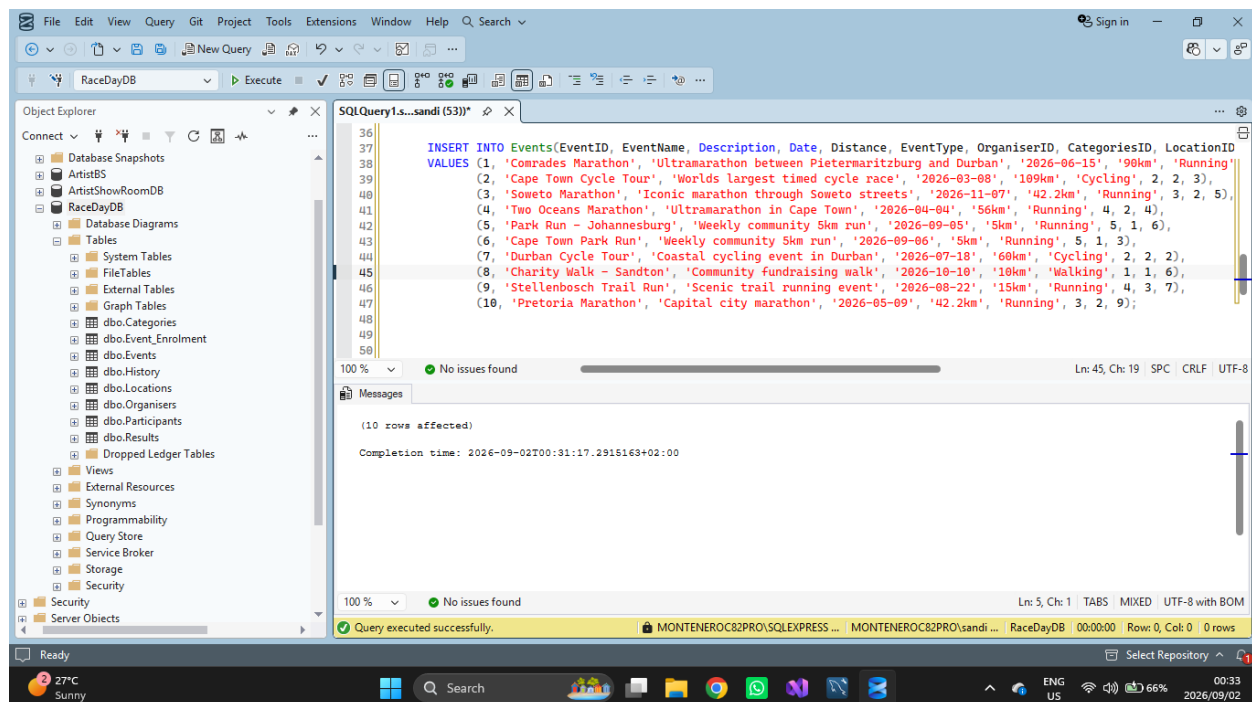
The SQL query you have used is able to insert participant records for five participants on the Participants table. Each of the participants was given a unique ParticipantID; First Name; Surname; Username; Password. These participants will be the individuals utilizing the RaceDay platform to access, choose and enroll in events. As long as a participant registers for an account, such as Thabo Mokoena, he will be able to login to the RaceDay platform using his username (ThaboM12), and Password, and from there utilize his dashboard to view up-and-coming events, filter through events based upon Category for example Age-Based Categories, Under 20 Senior 10 km 21 km Distance Based Categories, Select an event they wish to participate in, Enroll in that event by selecting a Category that links them to the Event and that selected Category; after enrolling, a participant will also be able to review their enrollment history, review their own race results as soon as possible after the organizer has recorded finish time and position for the event, and make changes to their own user-defined data such as their name and/or password. Most importantly, a participant cannot create events, manage categories or collect results these duties are reserved solely for the organizers however they will have complete visibility into their past activity within the RaceDay Platform and their results once posted by the organizers.



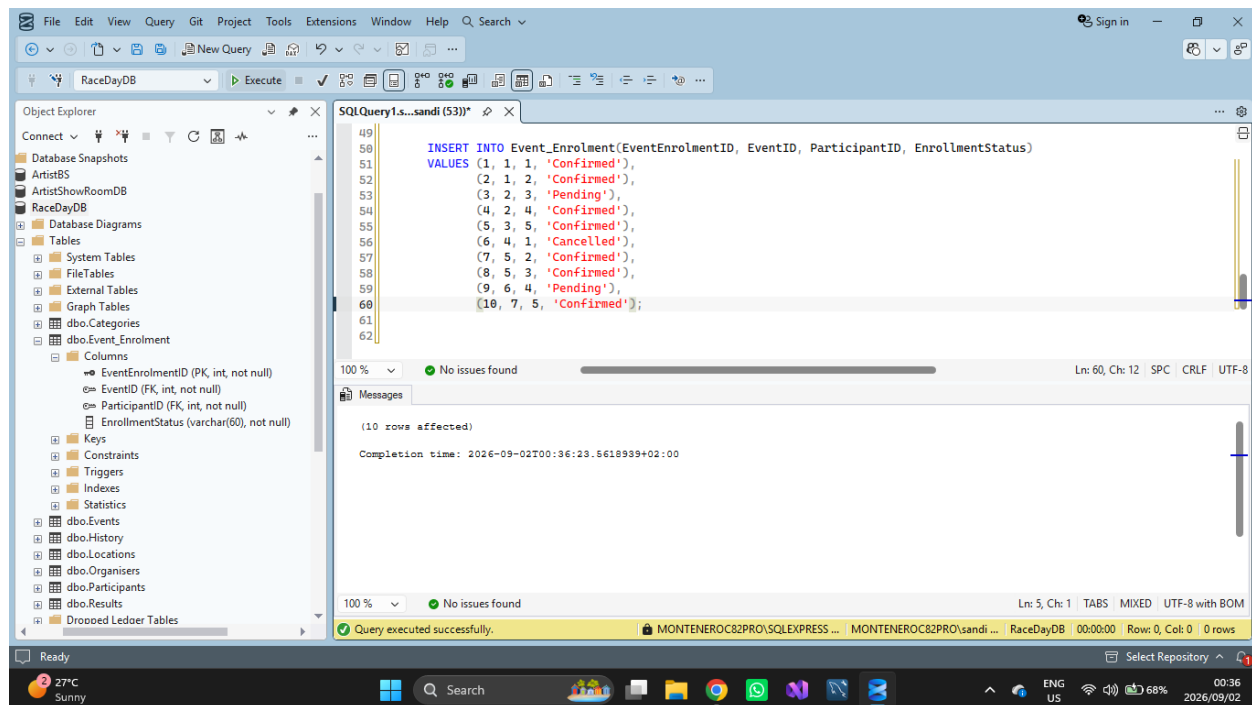
The SQL code you've supplied creates five category records in the Categories table, all having their own unique Categories ID (Categories ID), Age Specification, Category Type, Minimum Km Range and Maximum Km Range .The categories are used to divide participants by class or age group at events. A junior athlete may therefore be classified as 'Junior' with a Km distance of 5-10Km, whilst a 40 year old athlete may be classified as 'Master' with Km distances of 20-40Km - the categories then enable the organisers to allocate participants to the appropriate groups for the event enabling fair competition for all participants. The categories are defined and administered by the event administrator prior to the commencement of the event; the categories are then associated to the event via the Categories ID foreign key in the EVENTS table which enables each event to utilise their own categories depending upon the type of race format utilised. Upon enrollment to an event, participants select one of the categories available to them and this selection is recorded in the EVENT_ENROLMENT table along with the participant, event and selected category thereby enabling the administrators to capture results for each category individually for example determine who was first place in the 'Seniors' category or 'Masters' category. In summary categories provide flexibility and inclusiveness in accommodating participants of varying ages and fitness levels and are therefore fundamental to the RaceDay System.



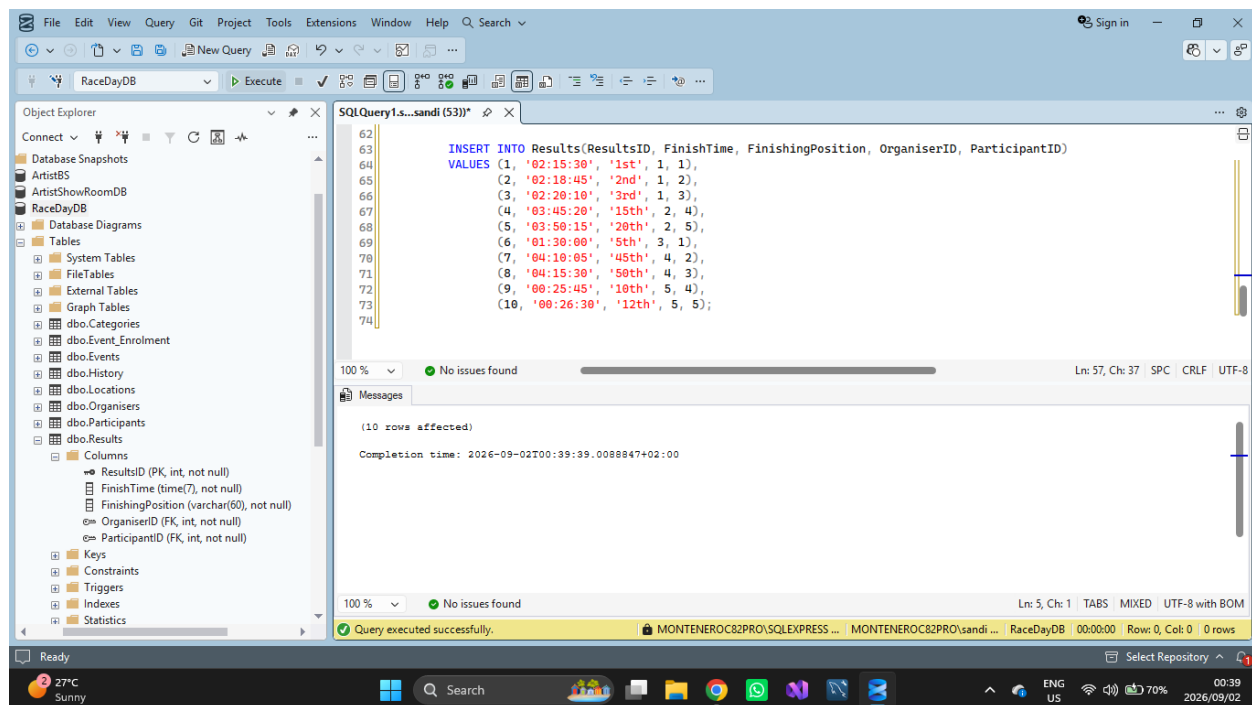
The purpose of your SQL code is to add ten locations into the Locations table and populate them with unique LocationIDs, provinces, towns, and addresses. These are representative of the numerous venues around South Africa at which races can occur. They include those such as Pietermaritzburg and Durban in KwaZulu Natal, Cape Town and Stellenbosch in the Western Cape, Soweto, Johannesburg, and Pretoria in Gauteng as well as Port Elizabeth in the Eastern Cape and Paarl in the Western Cape. Each location is represented by a single entry into the Locations table allowing the Organisers of Events to choose from this predefined selection of locations and have Race Day automatically associate the Event created with the selected location using the Location ID as a foreign key reference in the EVENTS table. This model enables the use of location information only once within Race Day as opposed to entering redundant information thereby keeping the database consistent and eliminating unnecessary duplication of effort. For instance, if Cape Town Stadium located on Beach Rd is going to be used as a hosting venue for the Cape Town Cycle Tour, Two Oceans Marathon and more, there is no need to enter the complete address every time. Because each location has been placed into its own table, Race Day provides organisers with an efficient method of managing updates to venue details in one area and provides participants with all of the necessary details about where an event will take place for example Province, City name and Physical Address, enabling them to make appropriate plans for transportation and accommodations.



The SQL statement used to populate the EVENTS TABLE will insert ten EVENT RECORDS with unique EVENT ID's, names for the events, descriptions for the events, the date of the event, how far the event is, the type of event and references to the organiser (foreign key), category (foreign key), and location (foreign key). These are the core elements of the Raceday System and cover an extensive range of different types of running, cycling and walking events throughout South Africa. For instance, the two most famous races in South Africa are the Comrades Marathon and the Two Oceans Marathon, however there are also many other community based events such as park runs and charity walks. Every event record is associated with a specific organiser through the use of an Organiser ID foreign key. This provides the organiser the ability to control, edit, see enrollments and enter race results related to the event. Furthermore, every event record is assigned both a Category ID and a Location ID foreign key. Therefore, every event record contains information about a pre-defined set of divisions age-based or distance-based as well as a defined physical venue at which the event will take place. Thus, this makes it easier for participants to know what the event entails and where they have to attend. As shown by the separation of event details from organizer, category and location detail within separate tables, this design reduces redundancy and insures data integrity. If the street address of the Cape Town stadium is changed then only one entry needs to be made in the Locations Table; all event entries referencing that same location will now contain the correct information. Overall, the Events Table acts as the main hub of the entire Raceday Platform allowing organizers to add/manage races, participants to search/browse/enroll in events and the entire system to monitor activity from when an event is created until when results are published.

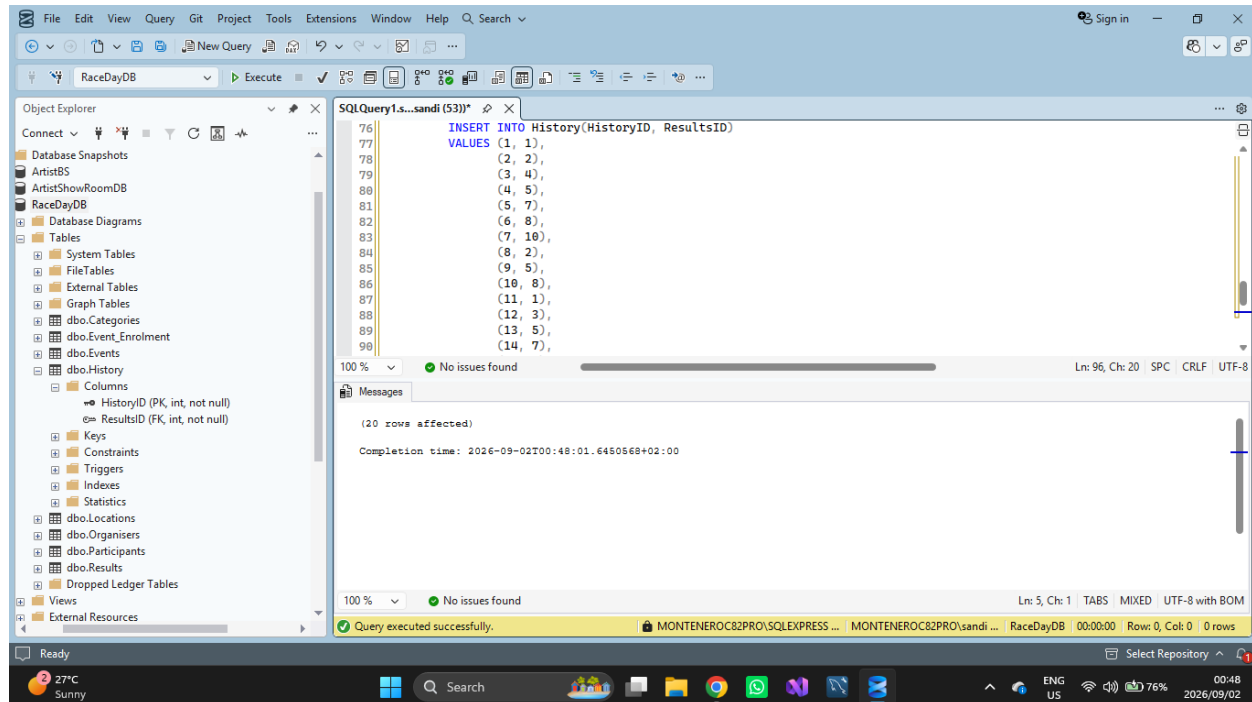


The Event_Enrolment Table represents the Operational Core of race day. It contains 10 Enrolments with a Unique EventEnrolmentID for each, along with EventID, ParticipantID and Enrolment Status. This creates a bridge between participants and events they have registered for. An Example is shown where Thabo Mokoena (ParticipantID 1) has enrolled for Comrades Marathon (Event ID 1) - Confirmed, Lerato Mthembu (ParticipantID 2) is also Confirmed for this event - Comrades Marathon (Event ID 1). Sipho Zulu (ParticipantID 3) has a Pending status for the Cape Town Cycle Tour (EventID 2), thus the Organiser has not accepted this Registration. The Enrolment Statuses - either Confirmed, Pending, or Cancelled allows Organisers to Manage Participant Registrations by Accepting or Rejecting Entries. The Enrolment Status will give the Participant Clarity on if their Registration has been Finalized. Upon Confirming an Enrolment the Organiser may then Capture a Result for that Specific Enrolment and Link the Participant's Finish Time and Position to the Event and Category the Participant Chose to Participate in. Furthermore the Enrolment Record is Key to Building the Participants Personal Performance History.



The SQL statement you've given adds ten result entries into the Results table. Each entry has a unique ResultsID, a FinishTime, a FinishingPosition, an OrganizerID and a ParticipantID. These entries represent the actual results from races once they're completed. They contain all the information about each competitor's finish time, and the position he/she achieved. As shown in the first entry, Sandile Nhlapo (OrganizerID = 1) managed the Comrades Marathon (the race itself); Thabo Mokoena (ParticipantID = 1) participated in the race. He took 01h 15m 30s to complete it and therefore secured 1st position. This is just one example of what the Results table could hold. It will contain hundreds of such examples when populated completely. The Results table contains two types of foreign keys: ParticipantsID and OrganizerID. OrganizerID will restrict to only those individuals who are allowed to manage an Event. Therefore, if someone attempts to enter incorrect data into the Results table for an event that does not belong to them, that action will fail. Data integrity will be maintained at all times. ParticipantID is used to link each participant's result to his own personal database entry, which allows him to monitor their own performances over time. Finally the FinishingPosition field contains values in string format for example '1st', '2nd' for easy viewing purposes. However since there is no way to sort these values using standard methods for example sorting alphabetically, in a more scalable solution it may be better to store the integer value of the finishing position. With this integer representation, you'll easily be able to perform the necessary calculations required to determine race statistics for example number of top three finishers per age group. Once entered the results will appear on participants' dashboards where they can

see how they've progressed throughout time, how they compared against other competitors in past races and create an extensive historical record of their accomplishments. Likewise organizers will be able to utilize this data to create reports, award prizes and create published results/rankings for races.



The SQL code in your question will enter twenty records into the History table. Each HistoryID is uniquely related to one ResultsID. These entries will allow you to document the historical performances of participants over time; for example produce a history of their race results. For example the first HistoryID (HistoryID = 1) has a relation to the first ResultsID (ResultsID = 1). This corresponds to Thabo Mokoena finishing the Comrades Marathon in 1st place. On the other hand, there is another reference from HistoryID = 8 to ResultsID = 2. Therefore, Lerato Mthembu finished the Comrades Marathon in second place, but she has two references in the History table either due to an error on behalf of the user entering the data twice or because he wants to archive her race results for two different years/events.

However, having the same ResultsID referenced multiple times within the History table does not represent duplication in the traditional way. Since the History table is specifically intended to provide historical views of results at any point in time, it may have repeated references to a particular result based upon how many times it was needed during that period. For example, archiving a summary of end of year results, documenting awards

given out during that season, etc. Historically speaking, regardless of whether a Result was altered or removed in the Results Table, its history will remain intact for audit and reporting purposes. Using this example again; if an organizer incorrectly entered a finish time and then corrected it, the initial finish time entry would be kept in the History table allowing for data to be transparently audited and traced back to its origin.

In summary the History table provides an unalterable and permanent time-stamp record of all Results captured since inception. As a result it allows users to generate long-term performance trend graphs, identify past winners, and view participants' racing history over their participation on the platform.

Full SQL Query

```
CREATE DATABASE RaceDayDB;
```

```
CREATE TABLE Organisers (  
  OrganiserID INT NOT NULL PRIMARY KEY,  
  ORname VARCHAR(60) NOT NULL,  
  ORsurname VARCHAR(60) NOT NULL,  
  ORusername VARCHAR(60) NOT NULL,  
  ORpassword VARCHAR(60) NOT NULL  
);
```

```
CREATE TABLE Participants (  
  ParticipantID INT NOT NULL PRIMARY KEY,  
  Pname VARCHAR(60) NOT NULL,  
  Psurname VARCHAR(60) NOT NULL,  
  Pusername VARCHAR(60) NOT NULL,  
  Ppassword VARCHAR(60) NOT NULL  
);
```

```
CREATE TABLE Categories (  
  CategoriesID INT NOT NULL PRIMARY KEY,  
  age INT NOT NULL,  
  CategoryType VARCHAR(60) NOT NULL,  
  MinKM INT NOT NULL,  
  MaxKM INT NOT NULL  
);
```

```
CREATE TABLE Locations (  
  LocationID INT NOT NULL PRIMARY KEY,  
  Province VARCHAR(60) NOT NULL,  
  Town VARCHAR(60) NOT NULL,  
  PhysicalAddress VARCHAR(255) NOT NULL  
);
```

```
CREATE TABLE Events (  
  EventID INT NOT NULL PRIMARY KEY,  
  EventName VARCHAR(60) NOT NULL,  
  Description VARCHAR(60) NOT NULL,  
  Date DATE NOT NULL,  
  Distance VARCHAR(60) NOT NULL,  
  EventType VARCHAR(60) NOT NULL,  
  OrganiserID INT NOT NULL,  
  CategoriesID INT NOT NULL,  
  LocationID INT NOT NULL,  
  CONSTRAINT FK_Events_Organisers FOREIGN KEY (OrganiserID) REFERENCES Organisers(OrganiserID),  
  CONSTRAINT FK_Events_Categories FOREIGN KEY (CategoriesID) REFERENCES Categories(CategoriesID),  
  CONSTRAINT FK_Events_Locations FOREIGN KEY (LocationID) REFERENCES Locations(LocationID)  
);
```

```
CREATE TABLE Event_Enrolment (  
  EventEnrolmentID INT NOT NULL PRIMARY KEY,  
  EventID INT NOT NULL,  
  ParticipantID INT NOT NULL,  
  EnrollmentStatus VARCHAR(60) NOT NULL,  
  CONSTRAINT FK_EventEnrolment_Events FOREIGN KEY (EventID) REFERENCES Events(EventID),  
  CONSTRAINT FK_EventEnrolment_Participants FOREIGN KEY (ParticipantID) REFERENCES Participants(ParticipantID)  
);
```

```
CREATE TABLE Results (  
  ResultsID INT NOT NULL PRIMARY KEY,  
  FinishTime TIME(7) NOT NULL,  
  FinishingPosition VARCHAR(60) NOT NULL,  
  OrganiserID INT NOT NULL,  
  ParticipantID INT NOT NULL,  
  CONSTRAINT FK_Results_Organisers FOREIGN KEY (OrganiserID) REFERENCES Organisers(OrganiserID),  
  CONSTRAINT FK_Results_Participants FOREIGN KEY (ParticipantID) REFERENCES Participants(ParticipantID)  
);
```

```
CREATE TABLE History (
    HistoryID INT NOT NULL PRIMARY KEY,
    ResultsID INT NOT NULL,
    CONSTRAINT FK_History_Results FOREIGN KEY (ResultsID) REFERENCES Results(ResultsID)
);
```

```
INSERT INTO Organisers(OrganiserID,ORname,ORSurname,ORusername,ORpassword)
VALUES(1,'Sandile','Nhlapo','sandile081','Sandilenhlapo11'),
(2,'Blessing','Mashinini','blessingB071','Bless44'),
(3,'Mandlakhe','Sgeenge','mandla67','Mwilliam'),
(4,'happyness','Masuku','happynessM','IamHappy'),
(5,'Katileho','Ndlovu','kat112','KGNdlovu');
```

```
INSERT INTO Participants(ParticipantID, Pname, Psurname, Pusername, Ppassword)
VALUES (1, 'Thabo', 'Mokoena', 'thaboM12', 'Thabo@2024'),
(2, 'Lerato', 'Mthembu', 'leratoL', 'Lerato#99'),
(3, 'Sipho', 'Zulu', 'siphoZ', 'SiphoZulu01'),
(4, 'Nomsa', 'Dlamini', 'nomsaD', 'Nomsa@Dla'),
(5, 'Kagiso', 'Petersen', 'kagisoP', 'Kagi$o2024');
```

```
INSERT INTO Categories(CategoriesID, age, CategoryType, MinKM, MaxKM)
VALUES (1, 18, 'Junior','5KM', '10KM'),
(2, 25, 'Open','10KM','15KM'),
(3, 35, 'Senior','15KM','30KM'),
(4, 40, 'Masters','20KM','40KM'),
(5, 50, 'Veteran','10KM','20KM');
```

```
INSERT INTO Locations(LocationID, Province, Town, PhysicalAddress)
VALUES (1, 'KwaZulu-Natal', 'Pietermaritzburg', '221 Alan Paton Avenue, Pietermaritzburg'),
(2, 'KwaZulu-Natal', 'Durban', '45 Masabalala Yengwa Avenue, Durban'),
(3, 'Western Cape', 'Cape Town', '12 Beach Road, Mouille Point, Cape Town'),
(4, 'Western Cape', 'Cape Town', '88 Main Road, Newlands, Cape Town'),
(5, 'Gauteng', 'Soweto', '1 Vilakazi Street, Soweto, Johannesburg'),
(6, 'Gauteng', 'Johannesburg', '44 Central Avenue, Sandton, Johannesburg'),
(7, 'Western Cape', 'Stellenbosch', '15 Dorp Street, Stellenbosch'),
(8, 'Eastern Cape', 'Port Elizabeth', '23 Marine Drive, Summerstrand, Port Elizabeth'),
(9, 'Gauteng', 'Pretoria', '78 Church Street, Pretoria Central'),
(10, 'Western Cape', 'Paarl', '5 Main Road, Paarl');
```

```
INSERT INTO Events(EventID, EventName, Description, Date, Distance, EventType, OrganiserID, CategoriesID,
```

LocationID)

VALUES (1, 'Comrades Marathon', 'Ultramarathon between Pietermaritzburg and Durban', '2026-06-15', '90km', 'Running', 1, 2, 1),

(2, 'Cape Town Cycle Tour', 'Worlds largest timed cycle race', '2026-03-08', '109km', 'Cycling', 2, 2, 3),

(3, 'Soweto Marathon', 'Iconic marathon through Soweto streets', '2026-11-07', '42.2km', 'Running', 3, 2, 5),

(4, 'Two Oceans Marathon', 'Ultramarathon in Cape Town', '2026-04-04', '56km', 'Running', 4, 2, 4),

(5, 'Park Run - Johannesburg', 'Weekly community 5km run', '2026-09-05', '5km', 'Running', 5, 1, 6),

(6, 'Cape Town Park Run', 'Weekly community 5km run', '2026-09-06', '5km', 'Running', 5, 1, 3),

(7, 'Durban Cycle Tour', 'Coastal cycling event in Durban', '2026-07-18', '60km', 'Cycling', 2, 2, 2),

(8, 'Charity Walk - Sandton', 'Community fundraising walk', '2026-10-10', '10km', 'Walking', 1, 1, 6),

(9, 'Stellenbosch Trail Run', 'Scenic trail running event', '2026-08-22', '15km', 'Running', 4, 3, 7),

(10, 'Pretoria Marathon', 'Capital city marathon', '2026-05-09', '42.2km', 'Running', 3, 2, 9);

INSERT INTO Event_Enrolment(EventEnrolmentID, EventID, ParticipantID, EnrollmentStatus)

VALUES (1, 1, 1, 'Confirmed'),

(2, 1, 2, 'Confirmed'),

(3, 2, 3, 'Pending'),

(4, 2, 4, 'Confirmed'),

(5, 3, 5, 'Confirmed'),

(6, 4, 1, 'Cancelled'),

(7, 5, 2, 'Confirmed'),

(8, 5, 3, 'Confirmed'),

(9, 6, 4, 'Pending'),

(10, 7, 5, 'Confirmed');

INSERT INTO Results(ResultsID, FinishTime, FinishingPosition, OrganiserID, ParticipantID)

VALUES (1, '02:15:30', '1st', 1, 1),

(2, '02:18:45', '2nd', 1, 2),

(3, '02:20:10', '3rd', 1, 3),

(4, '03:45:20', '15th', 2, 4),

(5, '03:50:15', '20th', 2, 5),

(6, '01:30:00', '5th', 3, 1),

(7, '04:10:05', '45th', 4, 2),

(8, '04:15:30', '50th', 4, 3),

(9, '00:25:45', '10th', 5, 4),

(10, '00:26:30', '12th', 5, 5);

INSERT INTO History(HistoryID, ResultsID)

VALUES (1, 1),

(2, 2),

(3, 4),

(4, 5),

(5, 7),
(6, 8),
(7, 10),
(8, 2),
(9, 5),
(10, 8),
(11, 1),
(12, 3),
(13, 5),
(14, 7),
(15, 9),
(16, 1),
(17, 3),
(18, 5),
(19, 7),
(20, 9);

